

SQL Cheatsheet

DDL: Database & Table Setup

Create / Drop Database

```
CREATE DATABASE sales;
USE sales;
DROP DATABASE IF EXISTS sales;
```

Create Table with Constraints

```
CREATE TABLE employees (
  emp_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(200) UNIQUE,
  dept_id INT,
  salary DECIMAL(10,2) DEFAULT 0,
  join_date DATE,
  FOREIGN KEY (dept_id) REFERENCES
    departments(dept_id) ON DELETE SET NULL
);
```

ALTER: Modify Structure

```
ALTER TABLE employees
  RENAME COLUMN city TO location;
ALTER TABLE t ADD COLUMN col INT;
ALTER TABLE t DROP COLUMN col;
ALTER TABLE t MODIFY COLUMN col VARCHAR(50);
```

INSERT

```
INSERT INTO stores(store_id) VALUES (3);
INSERT INTO employees
VALUES (1, 'Sam', 's@x.com', 2, 50000, '2024-01-01');
-- Bulk insert
INSERT INTO employees(name, email) VALUES
('Alice', 'a@x.com'), ('Bob', 'b@x.com');
```

Constraints Summary

Constraint	Meaning
NOT NULL	Must have a value
UNIQUE	No duplicate values
PRIMARY KEY	Not Null + Unique(one per table)
FOREIGN KEY	References another table's PK
DEFAULT	Fallback value if none given
CHECK	Validate with a condition

DML: Data Manipulation

```
UPDATE customers SET name='sam' WHERE email='aa';
DELETE FROM customers WHERE email='aa';

-- TRUNCATE: faster, resets auto increment (DDL)
TRUNCATE TABLE customers;
```

DELETE vs TRUNCATE vs DROP

DELETE: DML, WHERE allowed, rollbackable, slow.
TRUNCATE: DDL, removes all rows, resets identity.
DROP: DDL, removes entire table + structure.

SELECT & Filtering

Basic SELECT

```
SELECT DISTINCT city FROM customers;
SELECT customer_id, email FROM dim_customer LIMIT 15;
SELECT product_name AS 'prod_name' FROM dim_product;
```

WHERE Conditions

```
SELECT * FROM dim_customer WHERE gender = 'F';
SELECT * FROM dim_customer WHERE (gender = 'F')
  AND ((country = 'France') OR (date > '2022-01-01'))
-- IN / BETWEEN
SELECT * FROM orders WHERE order_no IN ('2','4');
SELECT * FROM orders WHERE amount BETWEEN 1 AND 5;
```

LIKE: Pattern Matching

% any number of chars, _ exactly one char

```
WHERE first_name LIKE 'T%';      -- starts with T
WHERE first_name LIKE 'T__f%y';  -- T, 2 chars, f...y
WHERE email NOT LIKE '%@gmail%'; -- exclude gmail
```

ORDER BY & GROUP BY

```
SELECT * FROM dim_product ORDER BY unit_price DESC;
SELECT
  category,
  COUNT(*) AS cnt,
  AVG(unit_price) AS avg_price,
  SUM(unit_price) AS total_price
FROM
  dim_product
GROUP BY
  category
HAVING
  avg_price > 500;
```

WHERE filters rows **before** grouping.
HAVING filters groups **after** aggregation.

JOINS & UNIONS

JOIN Types

```
-- INNER: only matching rows in both tables
SELECT * FROM orders o
INNER JOIN customers c ON o.cust_id = c.id;
-- LEFT: all left rows, NULLs for right misses
SELECT * FROM orders o
LEFT JOIN customers c ON o.cust_id = c.id;
-- RIGHT: all right rows
SELECT * FROM orders o
RIGHT JOIN customers c ON o.cust_id = c.id;
-- CROSS JOIN: cartesian product
SELECT * FROM colors CROSS JOIN sizes;
-- SELF JOIN: table joined with itself
SELECT e.name, m.name AS manager FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

JOIN	Result
INNER	Only matching rows
LEFT	All left + matched right (NULL if none)
RIGHT	All right + matched left (NULL if none)
FULL	All from both (emulate via UNION)
CROSS	Cartesian product (no ON clause)
SELF	Table joined with itself

UNION

```
-- FULL JOIN emulation (MySQL has no FULL JOIN)
SELECT * FROM orders o
LEFT JOIN customers c ON o.cust_id=c.id
UNION
SELECT * FROM orders o
RIGHT JOIN customers c ON o.cust_id=c.id;
```

UNION removes duplicates. Use UNION ALL to keep them.
Both queries must have same column count & compatible types.

NULL Handling

```
-- IS NULL / IS NOT NULL (never use = NULL)
SELECT * FROM employees WHERE dept_id IS NULL;
-- IFNULL: replace NULL with a default
SELECT IFNULL(salary, 0) FROM employees;
-- NULLIF: returns NULL when two values are equal
SELECT NULLIF(score, 0) FROM results;
```

- NULL = NULL evaluates to NULL, not TRUE
- Any arithmetic with NULL returns NULL
- COUNT(*) counts all rows; COUNT(col) skips NULLs
- DISTINCT treats multiple NULLs as one

Subqueries & EXISTS

```
-- Scalar subquery in WHERE
SELECT * FROM dim_product
WHERE unit_price > (SELECT AVG(unit_price) FROM
  dim_product);
-- EXISTS: true if subquery returns any row (fast)
SELECT * FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.cust_id
  = c.id);
-- NOT EXISTS: customers with no orders
SELECT * FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.
  cust_id = c.id);
```

Prefer EXISTS over IN for large correlated subqueries.
IN loads all values; EXISTS short circuits on first match.

CTEs (Common Table Expressions)

A Common Table Expression (CTE) is a **temporary result set** created using WITH. It exists only during the execution of a single query and makes complex SQL easier to read and organize.

1. Single CTE

```
WITH above_avg AS (
  SELECT *
  FROM dim_product
  WHERE unit_price >
    (SELECT AVG(unit_price) FROM dim_product)
)
SELECT *
FROM above_avg;
```

2. Multiple CTEs

```
WITH above_avg AS (
  SELECT *
  FROM dim_product
  WHERE unit_price >
    (SELECT AVG(unit_price) FROM dim_product)
),
filtered AS (
  SELECT *
  FROM above_avg
  WHERE product_name IN ('Figure Method', 'Huge
    Change')
)

SELECT *
FROM filtered
WHERE product_name = 'Figure Method';
```

Transformations

Numeric Functions

```
SELECT ROUND(unit_price, 2),
  FLOOR(unit_price), CEIL(unit_price),
  ABS(-5), MOD(10,3), POWER(2,10)
FROM dim_product;
```

String Functions

```
SELECT
  CONCAT(first_name, ' ', last_name),
  CONCAT_WS(' ', first_name, last_name, country),
  LENGTH(country), UPPER(city), LOWER(city),
  SUBSTRING(email, 1, 6),
  REPLACE(email, '@', '%'),
  LEFT(country,3), RIGHT(country,3),
  REVERSE(country), REPEAT(first_name,2),
  TRIM(' hello '),
  LPAD('5', 3, '0') -- '005'
FROM dim_customer;
```

Date Functions

```
SELECT NOW(), CURDATE(), CURTIME();
SELECT YEAR(d), MONTH(d), DAY(d),
  DAYNAME(d), WEEKDAY(d), -- 0=Mon
  DATEDIFF('2025-01-10', '2025-01-01'), -- 9
  ADDDATE(d, 30),
  DATE_FORMAT(d, '%W %M %e %Y')
FROM dim_date;
SELECT CAST('2025-01-01' AS DATETIME); -- type cast
```

CASE: Conditionals

```
SELECT *,
  CASE
    WHEN unit_price <= 100 THEN 'affordable'
    WHEN unit_price <= 200 THEN 'normal'
    ELSE 'expensive'
  END AS price_category
FROM dim_product;
```

Window Functions

Running Total: OVER

```
SELECT *,
  SUM(unit_price) OVER(ORDER BY launch_date) AS
    running_total,
  AVG(salary) OVER(PARTITION BY dept_id) AS dept_avg
FROM dim_product;
```

Frame Clauses

```
-- Explicit running total
SUM(price) OVER(ORDER BY date
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
-- Rolling 3 row average
AVG(price) OVER(ORDER BY date
  ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)
-- Grand total in every row
SUM(price) OVER(
  ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED
  FOLLOWING)
```

Ranking Functions

```
SELECT unit_price,
  ROW_NUMBER() OVER(ORDER BY unit_price),
  RANK() OVER(ORDER BY unit_price),
  DENSE_RANK() OVER(ORDER BY unit_price),
  NTILE(4) OVER(ORDER BY unit_price) AS
    quartile,
  PERCENT_RANK() OVER(ORDER BY unit_price)
FROM dim_product;
```

ROW_NUMBER()	Always unique; no ties
RANK()	Ties share rank, next rank skips
DENSE_RANK()	Ties share rank, no gap after
NTILE(n)	Divides rows into n equal buckets

PARTITION BY

```
SELECT unit_price, category,
       DENSE_RANK() OVER(
         PARTITION BY category ORDER BY unit_price DESC)
       AS cat_rank
FROM dim_product;
```

LAG & LEAD

```
SELECT *,
       LAG(temp, 1, 0) OVER(ORDER BY id) AS prev_day,
       LAG(temp, 2, 0) OVER(ORDER BY id) AS prev_2days,
       LEAD(temp, 1, 0) OVER(ORDER BY id) AS next_day,
       temp - LAG(temp,1) OVER(ORDER BY id) AS delta
FROM weather;
```

LAG(col, n, default) looks back n rows.
LEAD(col, n, default) looks forward n rows.
Third arg is the default when no row exists.

Real World Scenarios

Nth Highest Value

```
-- 2nd highest salary without window functions
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Nth highest using DENSE_RANK (general approach)
WITH ranked AS (
  SELECT *,
         DENSE_RANK() OVER (ORDER BY salary DESC)
         AS rk
  FROM employees
)
SELECT * FROM ranked WHERE rk = 2;
```

Remove / Find Duplicates

```
-- Find duplicate emails
SELECT email, COUNT(*) FROM customers
GROUP BY email HAVING COUNT(*) > 1;

-- Delete duplicates, keep first occurrence
DELETE FROM customers
WHERE id NOT IN (
  SELECT MIN(id) FROM customers GROUP BY email
);

-- Tag duplicates with ROW_NUMBER
WITH numbered AS (
  SELECT *, ROW_NUMBER()
           OVER ( PARTITION BY email ORDER BY id) AS rn
  FROM customers
)
SELECT * FROM numbered WHERE rn > 1;
```

Consecutive / Running Sequences

```
-- Employees with salary > previous employee
SELECT name, salary FROM (
  SELECT name, salary,
         LAG(salary) OVER(ORDER BY emp_id) AS prev_sal
  FROM employees
) t WHERE salary > prev_sal;

-- Gap detection (find missing IDs)
SELECT id+1 AS missing FROM t
WHERE id+1 NOT IN (SELECT id FROM t);
```

Department wise Top N

```
-- Top 3 earners per department
SELECT * FROM (
  SELECT *,
         DENSE_RANK() OVER(
           PARTITION BY dept_id ORDER BY salary DESC) AS
         rk
  FROM employees
) t WHERE rk <= 3;
```

Views

```
CREATE VIEW top_earners AS
SELECT * FROM (
  SELECT *, DENSE_RANK() OVER(ORDER BY salary DESC)
  AS rk
  FROM employees
) t WHERE rk <= 10;

SELECT * FROM top_earners; -- use like a table
DROP VIEW IF EXISTS top_earners;
```

Views store the query, not the data.
Every SELECT re runs the underlying query.

Indexes

```
CREATE INDEX idx_email ON employees(email);
CREATE UNIQUE INDEX idx_uq ON employees(email);
DROP INDEX idx_email ON employees;
-- Composite index (leftmost prefix rule)
CREATE INDEX idx_dept_sal ON employees(dept_id,
salary);
```

Index Interview Points:

- Speeds SELECT, slows INSERT/UPDATE/DELETE
- MySQL uses B-Tree indexes by default
- Composite: order matters (leftmost prefix rule)
- Avoid indexing low cardinality cols (e.g., boolean)

Transactions & ACID

```
START TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE
id=1;
UPDATE accounts SET balance = balance + 500 WHERE
id=2;
COMMIT; -- make permanent

ROLLBACK; -- undo all since START TRANSACTION

SAVEPOINT sp1;
DELETE FROM orders WHERE status = 'cancelled';
ROLLBACK TO sp1; -- undo only back to savepoint
COMMIT;
```

ACID Properties:

Property	Guarantee
Atomicity	All-or-nothing; no partial updates
Consistency	DB moves from valid state to valid state
Isolation	Concurrent transactions don't interfere
Durability	Committed data persists after crash

Normalization

NF	Rule
1NF	Atomic values; no repeating groups; PK required
2NF	1NF + every non-key attr fully depends on whole PK
3NF	2NF + no transitive dependency (non-key → non-key)
BCNF	Stricter 3NF: every determinant is a candidate key

Stored Procedures & Functions

```
DELIMITER //
CREATE PROCEDURE add_employee(
  IN p_name CHAR(100), IN p_email CHAR(100))
BEGIN
  INSERT INTO employees(name, email) VALUES (p_name,
    p_email);
END //
DELIMITER ;
CALL add_employee('Sam','s@x.com');

DELIMITER //
CREATE FUNCTION square_it(x INT)
RETURNS INT DETERMINISTIC
BEGIN RETURN x * x; END //
DELIMITER ;
SELECT square_it(unit_price) FROM dim_product;
```

Function	Returns value used in SELECT
Procedure	No return, called with CALL
IN	Input parameter (read only inside)
OUT	Output parameter (write, read outside)
INOUT	Both read and written

Triggers

```
DELIMITER //
CREATE TRIGGER before_salary_update
BEFORE UPDATE ON employees FOR EACH ROW
BEGIN
  IF NEW.salary < 0 THEN
    SET NEW.salary = 0; -- clamp to 0
  END IF;
END //
DELIMITER ;
```

Timing: BEFORE or AFTER.

Events: INSERT, UPDATE, DELETE.

NEW = incoming values; OLD = previous values.

SQL Quick Reference

Query Execution Order

1. FROM / JOIN
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT (aliases created here)
6. DISTINCT
7. ORDER BY
8. LIMIT / OFFSET

Aggregate Functions

COUNT(*)	Count all rows
COUNT(col)	Count non-NULL values
SUM(col)	Sum of values
AVG(col)	Average (ignores NULL)
MIN(col)	Minimum value
MAX(col)	Maximum value
GROUP_CONCAT	Concatenate values